

# Refactoring Plan for 3D TDSE Programs:

## Modular Architecture for Strong-Field Ionization and High-Harmonic Generation

Based on Analysis of:  
`D_inner_out_volkov_3d_with_prob.f90`  
`rescatteing+hhg-he.f90`

November 21, 2025

### Abstract

This document presents a comprehensive refactoring plan for two Fortran programs that solve the time-dependent Schrödinger equation (TDSE) for atoms in strong laser fields. The programs implement the split-operator method with generalized pseudospectral (GPS) discretization on Legendre collocation grids. We propose a modular architecture that separates concerns into well-defined components: Hamiltonian construction, initial state preparation, time propagation, region splitting, observable extraction, and I/O management. The modular design enables both ionization (ATI) and high-harmonic generation (HHG) calculations within a unified framework while maintaining numerical efficiency and parallel scalability.

## Contents

|          |                                        |          |
|----------|----------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                    | <b>4</b> |
| 1.1      | Overview . . . . .                     | 4        |
| 1.2      | Refactoring Goals . . . . .            | 4        |
| 1.3      | Target Programming Languages . . . . . | 4        |
| <b>2</b> | <b>Architectural Overview</b>          | <b>5</b> |
| 2.1      | Module Hierarchy . . . . .             | 5        |
| 2.2      | Data Flow Diagram . . . . .            | 5        |
| <b>3</b> | <b>Module Specifications</b>           | <b>7</b> |
| 3.1      | Module 1: Grid Module . . . . .        | 7        |
| 3.1.1    | Purpose . . . . .                      | 7        |
| 3.1.2    | Mathematical Foundation . . . . .      | 7        |
| 3.1.3    | Interface . . . . .                    | 8        |
| 3.1.4    | Implementation Notes . . . . .         | 8        |
| 3.1.5    | Testing Strategy . . . . .             | 9        |
| 3.2      | Module 2: Hamiltonian Module . . . . . | 10       |
| 3.2.1    | Purpose . . . . .                      | 10       |
| 3.2.2    | Mathematical Foundation . . . . .      | 10       |
| 3.2.3    | Potential Models . . . . .             | 10       |
| 3.2.4    | Interface . . . . .                    | 11       |
| 3.2.5    | Implementation Notes . . . . .         | 11       |
| 3.2.6    | Testing Strategy . . . . .             | 11       |
| 3.3      | Module 3: Eigenstate Module . . . . .  | 12       |
| 3.3.1    | Purpose . . . . .                      | 12       |
| 3.3.2    | Mathematical Foundation . . . . .      | 12       |
| 3.3.3    | Interface . . . . .                    | 12       |

|        |                                                 |    |
|--------|-------------------------------------------------|----|
| 3.3.4  | Implementation Notes . . . . .                  | 12 |
| 3.3.5  | Testing Strategy . . . . .                      | 13 |
| 3.4    | Module 4: Field Module . . . . .                | 14 |
| 3.4.1  | Purpose . . . . .                               | 14 |
| 3.4.2  | Mathematical Foundation . . . . .               | 14 |
| 3.4.3  | Interface . . . . .                             | 15 |
| 3.4.4  | Implementation Notes . . . . .                  | 15 |
| 3.4.5  | Testing Strategy . . . . .                      | 15 |
| 3.5    | Module 5: Propagator Module . . . . .           | 16 |
| 3.5.1  | Purpose . . . . .                               | 16 |
| 3.5.2  | Mathematical Foundation . . . . .               | 16 |
| 3.5.3  | Interface . . . . .                             | 17 |
| 3.5.4  | Algorithm: One Time Step . . . . .              | 17 |
| 3.5.5  | Implementation Notes . . . . .                  | 17 |
| 3.5.6  | Testing Strategy . . . . .                      | 17 |
| 3.6    | Module 6: Region Split Module . . . . .         | 19 |
| 3.6.1  | Purpose . . . . .                               | 19 |
| 3.6.2  | Mathematical Foundation . . . . .               | 19 |
| 3.6.3  | Interface . . . . .                             | 19 |
| 3.6.4  | Physical Interpretation . . . . .               | 19 |
| 3.6.5  | Implementation Notes . . . . .                  | 20 |
| 3.6.6  | Testing Strategy . . . . .                      | 20 |
| 3.7    | Module 7: Continuum Projection Module . . . . . | 21 |
| 3.7.1  | Purpose . . . . .                               | 21 |
| 3.7.2  | Mathematical Foundation . . . . .               | 21 |
| 3.7.3  | Interface . . . . .                             | 22 |
| 3.7.4  | Algorithm: Momentum Projection . . . . .        | 22 |
| 3.7.5  | Implementation Notes . . . . .                  | 23 |
| 3.7.6  | Testing Strategy . . . . .                      | 23 |
| 3.8    | Module 8: Observable Module . . . . .           | 24 |
| 3.8.1  | Purpose . . . . .                               | 24 |
| 3.8.2  | Mathematical Foundation . . . . .               | 24 |
| 3.8.3  | Interface . . . . .                             | 25 |
| 3.8.4  | Implementation Notes . . . . .                  | 25 |
| 3.8.5  | Testing Strategy . . . . .                      | 25 |
| 3.9    | Module 9: I/O Module . . . . .                  | 26 |
| 3.9.1  | Purpose . . . . .                               | 26 |
| 3.9.2  | Input Parameters . . . . .                      | 26 |
| 3.9.3  | Output Files . . . . .                          | 27 |
| 3.9.4  | Interface . . . . .                             | 27 |
| 3.9.5  | Implementation Notes . . . . .                  | 27 |
| 3.9.6  | Testing Strategy . . . . .                      | 28 |
| 3.10   | Module 10: Orchestrator Module . . . . .        | 29 |
| 3.10.1 | Purpose . . . . .                               | 29 |
| 3.10.2 | Main Execution Flow . . . . .                   | 30 |
| 3.10.3 | Interface . . . . .                             | 31 |
| 3.10.4 | Error Handling . . . . .                        | 31 |
| 3.10.5 | Performance Optimization . . . . .              | 31 |

|           |                                                  |           |
|-----------|--------------------------------------------------|-----------|
| <b>4</b>  | <b>Integration Plan</b>                          | <b>32</b> |
| 4.1       | Phase 1: Foundation Layer (Weeks 1-2)            | 32        |
| 4.2       | Phase 2: Preparation Layer (Weeks 3-4)           | 32        |
| 4.3       | Phase 3: Evolution Layer (Weeks 5-7)             | 32        |
| 4.4       | Phase 4: Analysis Layer (Weeks 8-9)              | 33        |
| 4.5       | Phase 5: Application Layer (Week 10)             | 33        |
| 4.6       | Phase 6: Validation & Optimization (Weeks 11-12) | 33        |
| <b>5</b>  | <b>Testing Strategy</b>                          | <b>34</b> |
| 5.1       | Unit Tests                                       | 34        |
| 5.2       | Integration Tests                                | 34        |
| 5.3       | Regression Tests                                 | 34        |
| 5.4       | Performance Tests                                | 34        |
| <b>6</b>  | <b>Numerical Considerations</b>                  | <b>35</b> |
| 6.1       | Numerical Precision                              | 35        |
| 6.2       | Stability Criteria                               | 35        |
| 6.3       | Convergence Parameters                           | 35        |
| 6.4       | Parallelization Strategy                         | 35        |
| <b>7</b>  | <b>Code Structure Example</b>                    | <b>36</b> |
| 7.1       | Directory Layout                                 | 36        |
| 7.2       | Example Module Interface (Julia)                 | 37        |
| <b>8</b>  | <b>Mathematical Reference</b>                    | <b>38</b> |
| 8.1       | Key Equations Summary                            | 38        |
| 8.2       | Physical Constants (Atomic Units)                | 38        |
| 8.3       | Useful Relations                                 | 39        |
| <b>9</b>  | <b>Recommended Libraries and Tools</b>           | <b>40</b> |
| 9.1       | Julia Ecosystem                                  | 40        |
| 9.2       | Python Ecosystem                                 | 40        |
| 9.3       | C++ Ecosystem                                    | 40        |
| <b>10</b> | <b>Migration Path from Fortran</b>               | <b>41</b> |
| 10.1      | Step-by-Step Migration                           | 41        |
| 10.2      | Fortran-to-Target Language Mapping               | 41        |
| 10.3      | Common Pitfalls                                  | 41        |
| <b>11</b> | <b>Example Workflow</b>                          | <b>42</b> |
| 11.1      | Ionization Calculation                           | 42        |
| 11.2      | HHG Calculation                                  | 42        |
| <b>12</b> | <b>Future Extensions</b>                         | <b>43</b> |
| 12.1      | Potential Enhancements                           | 43        |
| 12.2      | Code Organization for Extensions                 | 43        |
| <b>13</b> | <b>Conclusion</b>                                | <b>44</b> |

# 1 Introduction

## 1.1 Overview

The original Fortran codes implement sophisticated algorithms for solving the 3D TDSE in spherical coordinates using:

- **Split-operator time propagation** in energy representation
- **Generalized pseudospectral (GPS) method** with algebraic coordinate mapping
- **Region splitting** with smooth transition functions
- **Projection onto continuum states** (Volkov or Coulomb-Volkov)
- **3D sequential transformations** ( $\text{radial} \rightarrow \theta \rightarrow \phi$ )

## 1.2 Refactoring Goals

1. **Modularity:** Separate distinct physical/numerical operations into independent modules
2. **Maintainability:** Clear interfaces and documentation
3. **Extensibility:** Easy addition of new features (different atoms, field configurations)
4. **Performance:** Preserve numerical efficiency and parallel scalability
5. **Testability:** Unit tests for each module
6. **Unified framework:** Single codebase for both ionization and HHG calculations

## 1.3 Target Programming Languages

Recommended languages (in order of preference):

- **Julia:** High performance, easy parallel computing, good for scientific computing
- **Python + NumPy/Numba:** Rapid development, extensive libraries, reasonable performance
- **C++ with Eigen/Armadillo:** Maximum performance, more complex but production-ready
- **Fortran 2018+:** Modern Fortran with object-oriented features (if staying with Fortran)

## 2 Architectural Overview

### 2.1 Module Hierarchy

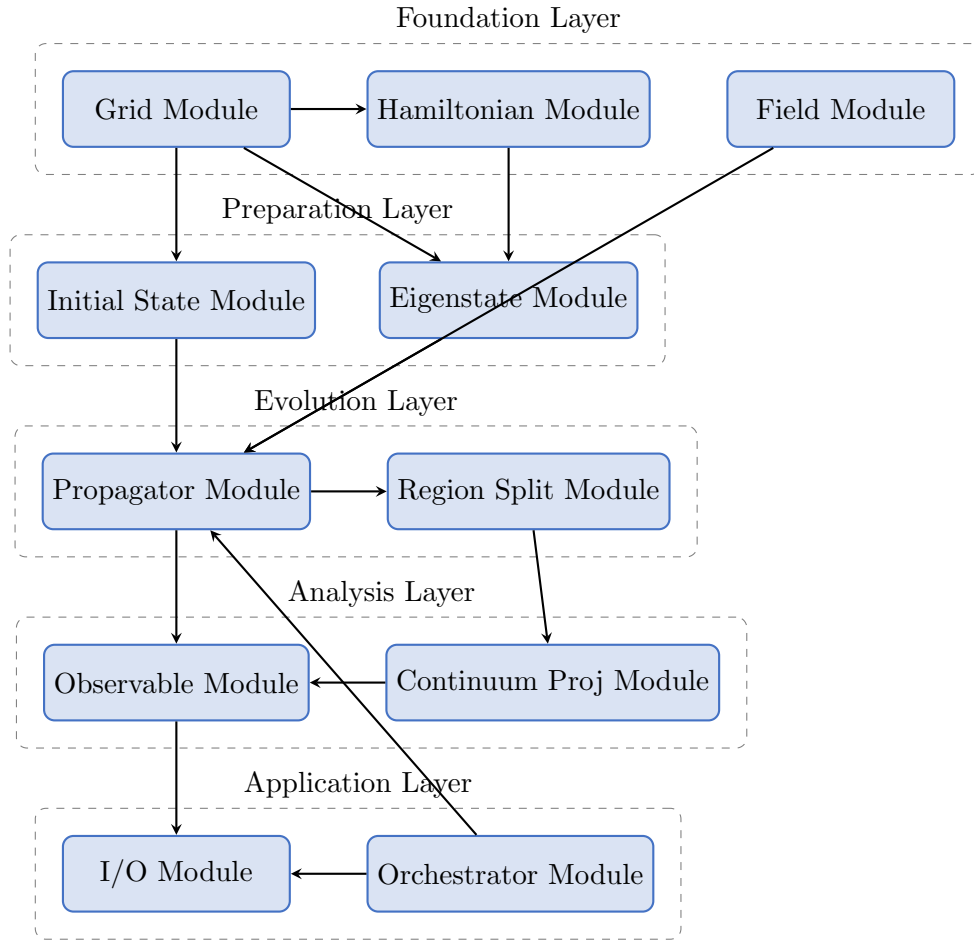


Figure 1: Modular architecture hierarchy. Arrows indicate data flow and dependencies.

### 2.2 Data Flow Diagram

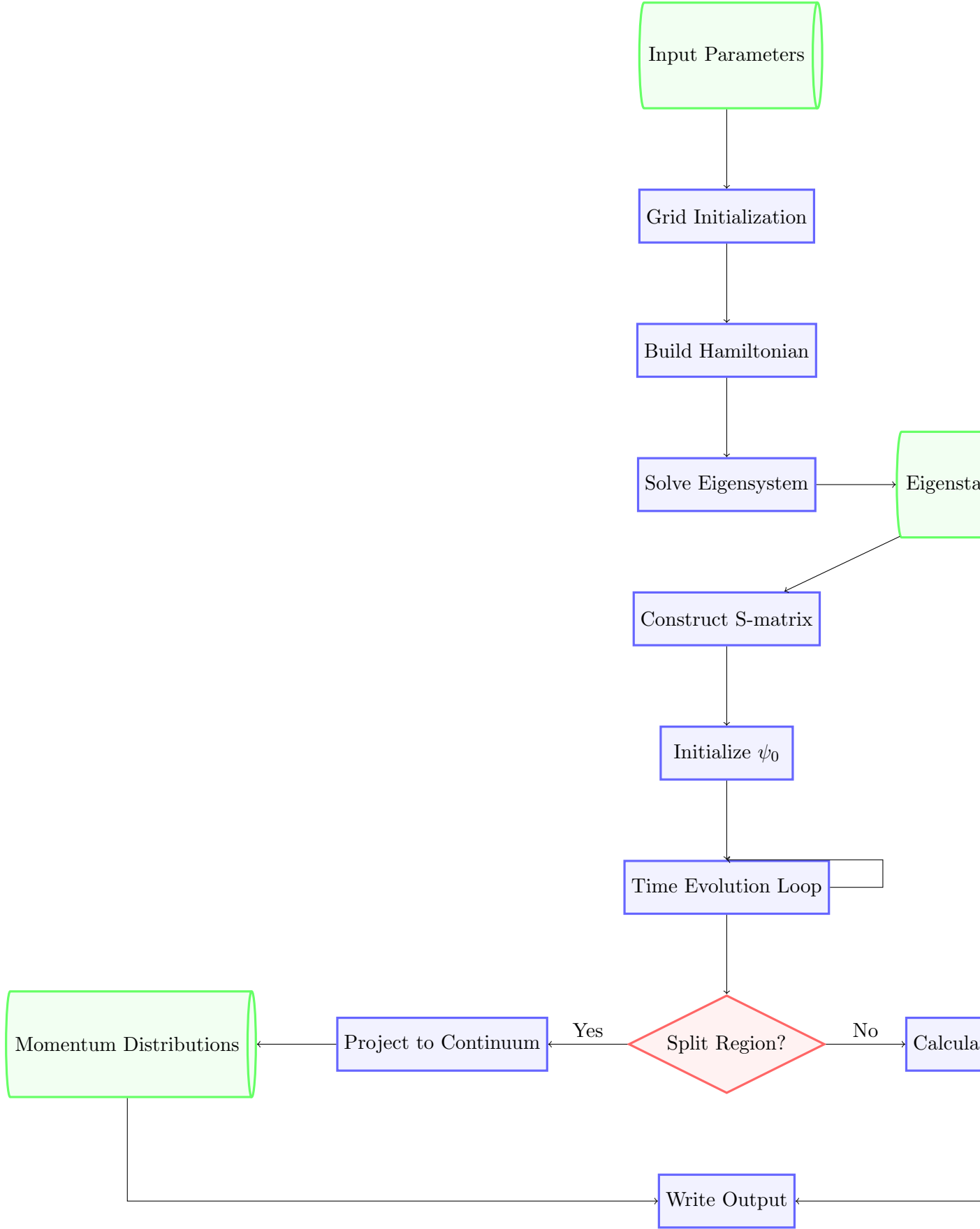


Figure 2: High-level data flow showing initialization, evolution, and observable extraction.

### 3 Module Specifications

#### 3.1 Module 1: Grid Module

##### 3.1.1 Purpose

Generate non-uniform spatial grids using the GPS (Generalized Pseudospectral) method with algebraic coordinate mapping.

##### 3.1.2 Mathematical Foundation

The GPS method uses Legendre collocation points with algebraic mapping:

$$r(x) = L \frac{1+x}{1-x+\alpha}, \quad x \in [-1, 1] \quad (1)$$

where  $x_i$  are zeros of the Legendre polynomial derivative  $P'_{N_r}(x)$ .

The mapping Jacobian is:

$$\frac{dr}{dx} = \frac{2L(1+\alpha)}{(1-x+\alpha)^2} \quad (2)$$

Angular grids use Legendre-Gauss quadrature:

$$\theta_j = \arccos(y_j), \quad y_j \in [-1, 1] \quad (3)$$

$$\phi_k = 2\pi k/N_\phi, \quad k = 0, \dots, N_\phi - 1 \quad (4)$$

### 3.1.3 Interface

#### Grid Module Interface

##### Inputs:

- $N_r$ : Number of radial grid points
- $N_\theta$ : Number of polar angle grid points
- $N_\phi$ : Number of azimuthal angle grid points
- $L$ : Mapping length parameter
- $\alpha$ : Mapping parameter
- $r_{max}$ : Maximum radius
- $l_{max}$ : Maximum angular momentum

##### Outputs:

- $\{x_i\}$ : Legendre collocation points
- $\{r_i\}$ : Mapped radial grid
- $\{r'_i\}$ : Mapping Jacobian values
- $\{w_i\}$ : Quadrature weights
- $\{\theta_j\}$ : Polar angle grid
- $\{w_\theta(j)\}$ : Theta quadrature weights
- $\{\phi_k\}$ : Azimuthal angle grid
- $\{P_n(x_i)\}$ : Legendre polynomials at grid points
- $\{P'_n(x_i)\}$ : Legendre polynomial derivatives
- $\{P_l^m(\cos \theta_j)\}$ : Associated Legendre polynomials

##### Methods:

- `compute_legendre_zeros()`: Find  $P'_{N_r}(x) = 0$
- `apply_mapping()`: Transform  $x \rightarrow r(x)$
- `compute_weights()`: Gauss-Legendre quadrature weights
- `generate_angular_grids()`:  $\theta$  and  $\phi$  discretization
- `compute_basis_functions()`:  $P_n(x_i)$ ,  $P_l^m(\cos \theta_j)$

### 3.1.4 Implementation Notes

- Use Newton-Raphson to find zeros of  $P'_{N_r}(x)$
- Store basis functions as sparse matrices if memory is limited
- Pre-compute and cache frequently used values (e.g.,  $e^{im\phi_k}$ )



- Implement parallel computation for basis function evaluation

### 3.1.5 Testing Strategy

- Verify zeros satisfy  $P'_{N_r}(x_i) = 0$  to machine precision
- Check orthogonality:  $\int_{-1}^1 P_n(x)P_m(x)dx = \frac{2}{2n+1}\delta_{nm}$
- Validate quadrature: test integration of polynomials up to degree  $2N_r - 1$
- Compare with analytical mappings for simple test cases

## 3.2 Module 2: Hamiltonian Module

### 3.2.1 Purpose

Construct the field-free atomic Hamiltonian in the GPS representation.

### 3.2.2 Mathematical Foundation

The Hamiltonian in atomic units (radial part for each  $l$ ):

$$\hat{H}_l = -\frac{1}{2} \frac{d^2}{dr^2} + \frac{l(l+1)}{2r^2} + V(r) \quad (5)$$

In the GPS representation with  $g(x) = \sqrt{r'(x)}u(r(x))$ :

$$H_{l,ij} = \int_{-1}^1 \left[ \frac{P'_{N_r}(x)}{P'_{N_r}(x_i)} \right]' \left[ \frac{P'_{N_r}(x)}{P'_{N_r}(x_j)} \right]' \frac{dx}{2[r'(x)]^2} + \left[ \frac{l(l+1)}{2r_i^2} + V(r_i) \right] \delta_{ij} \quad (6)$$

### 3.2.3 Potential Models

**Hydrogen:**

$$V(r) = -\frac{1}{r} \quad (7)$$

**Helium (single active electron):**

$$V(r) = -\frac{Z_{\text{eff}}}{r} e^{-r/r_D} \quad (8)$$

where  $Z_{\text{eff}} = 1.4$  and  $r_D = 17.85$  a.u. (Debye screening).

**Soft-core potential:**

$$V(r) = -\frac{Z}{\sqrt{r^2 + a^2}} \quad (9)$$

### 3.2.4 Interface

#### Hamiltonian Module Interface

##### Inputs:

- Grid data (from Grid Module)
- Atomic parameters:  $Z$ , nuclear charge; potential type
- $l_{max}$ : Maximum angular momentum

##### Outputs:

- $\{H_l\}_{l=0}^{l_{max}}$ : Hamiltonian matrices for each  $l$  block
- Dimensions:  $H_l \in \mathbb{R}^{N_r \times N_r}$

##### Methods:

- `construct_kinetic_matrix(1)`: Build kinetic energy operator
- `construct_potential_matrix()`: Diagonal potential term
- `construct_centrifugal_matrix(1)`:  $l(l+1)/(2r^2)$  term
- `assemble_hamiltonian(1)`: Combine all terms
- `get_hamiltonian_block(1)`: Retrieve  $H_l$

### 3.2.5 Implementation Notes

- Hamiltonian is symmetric:  $H_{ij} = H_{ji}$
- Typically sparse for large  $N_r$ ; consider sparse storage
- Each  $l$  block is independent: parallelize over  $l$
- Cache potential values  $V(r_i)$  to avoid repeated evaluations

### 3.2.6 Testing Strategy

- Verify Hermiticity:  $H = H^\dagger$
- Compare ground state energy with known values (e.g., H:  $-0.5$  Ha, He:  $-0.9$  Ha)
- Check eigenvalue convergence with increasing  $N_r$
- Validate matrix elements against analytical formulas for simple potentials

### 3.3 Module 3: Eigenstate Module

#### 3.3.1 Purpose

Solve the time-independent Schrödinger equation to obtain eigenstates and eigenvalues of the field-free Hamiltonian.

#### 3.3.2 Mathematical Foundation

For each angular momentum block:

$$H_l \vec{\phi}_{n,l} = E_{n,l} \vec{\phi}_{n,l} \quad (10)$$

The complete eigenstate is:

$$\Phi_{n,l,m}(r, \theta, \phi) = \phi_{n,l}(r) Y_l^m(\theta, \phi) \quad (11)$$

#### 3.3.3 Interface

##### Eigenstate Module Interface

###### Inputs:

- $\{H_l\}$  from Hamiltonian Module
- $N_{\text{bound}}$ : Number of bound states to compute
- $E_{\text{cutoff}}$ : Maximum energy for continuum states

###### Outputs:

- $\{E_{n,l}\}$ : Eigenvalues (energies)
- $\{\vec{\phi}_{n,l}\}$ : Eigenvectors (radial wavefunctions)
- Classification: bound ( $E < 0$ ) vs. continuum ( $E > 0$ )

###### Methods:

- `solve_eigensystem(1)`: Diagonalize  $H_l$
- `get_eigenvalue(n, 1)`: Retrieve  $E_{n,l}$
- `get_eigenstate(n, 1)`: Retrieve  $\vec{\phi}_{n,l}$
- `classify_states()`: Separate bound and continuum
- `normalize_states()`: Ensure  $\langle \phi_{n,l} | \phi_{m,l} \rangle = \delta_{nm}$

#### 3.3.4 Implementation Notes

- Use LAPACK/BLAS routines (e.g., `dsyev`, `dsyevd`) for diagonalization
- Each  $l$  block can be solved independently: parallelize
- Store only eigenstates with  $E_{n,l} < E_{\text{cutoff}}$  to save memory
- Normalization integral:

$$\int_0^\infty |\phi_{n,l}(r)|^2 dr = \sum_{i=1}^{N_r} |\phi_{n,l}(x_i)|^2 w_i r'(x_i) \quad (12)$$

### 3.3.5 Testing Strategy

- Compare ground state energy with literature values
- Verify orthonormality:  $\langle \phi_{n,l} | \phi_{m,l} \rangle = \delta_{nm}$
- Check degeneracies for hydrogen (expected from quantum numbers)
- Validate eigenfunction behavior near  $r = 0$  and  $r \rightarrow \infty$

### 3.4 Module 4: Field Module

#### 3.4.1 Purpose

Define and compute the time-dependent electric field  $\vec{E}(t)$ .

#### 3.4.2 Mathematical Foundation

General form with envelope function  $f(t)$ :

$$\vec{E}(t) = \vec{E}_0 f(t) \cos(\omega t + \phi) \quad (13)$$

Common envelope functions:

**Trapezoidal (programs use this):**

$$f(t) = \begin{cases} t/t_{\text{ramp}} & 0 \leq t < t_{\text{ramp}} \\ 1 & t_{\text{ramp}} \leq t < T - t_{\text{ramp}} \\ (T - t)/t_{\text{ramp}} & T - t_{\text{ramp}} \leq t \leq T \\ 0 & \text{otherwise} \end{cases} \quad (14)$$

**Sin-squared:**

$$f(t) = \sin^2\left(\frac{\pi t}{T}\right) \quad (15)$$

**Gaussian:**

$$f(t) = \exp\left[-4 \ln(2) \frac{(t - t_0)^2}{\tau^2}\right] \quad (16)$$

The vector potential (needed for gauge transformation):

$$\vec{A}(t) = - \int_0^t \vec{E}(t') dt' \quad (17)$$

### 3.4.3 Interface

#### Field Module Interface

##### Inputs:

- $E_0$ : Peak field strength
- $\omega$ : Laser frequency
- $\phi$ : Carrier-envelope phase
- Polarization vector:  $(\epsilon_x, \epsilon_y, \epsilon_z)$
- Envelope parameters:  $T$  (duration),  $t_{\text{ramp}}$ , etc.

##### Outputs:

- Functions to compute  $\vec{E}(t)$  and  $\vec{A}(t)$  on-demand for any time  $t$
- Derived quantities:  $U_p = E_0^2/(4\omega^2)$ ,  $\gamma = \sqrt{I_p/(2U_p)}$

##### Methods:

- `compute_envelope(t)`: Evaluate  $f(t)$
- `compute_electric_field(t)`: Evaluate  $\vec{E}(t)$
- `compute_vector_potential(t)`: Evaluate  $\vec{A}(t)$
- `get_field_parameters()`: Return  $U_p$ ,  $\gamma$ , etc.

### 3.4.4 Implementation Notes

- **Compute field on-the-fly**: Evaluating  $\vec{E}(t) = E_0 f(t) \cos(\omega t)$  is negligible cost ( $\sim 50$  CPU cycles) compared to S-matrix operations (millions). Pre-computation wastes memory without performance benefit.
- **Exception**: Only use lookup tables for experimental pulse data read from files
- For vector potential  $\vec{A}(t)$ : Use analytical integrals when possible (available for  $\sin^2$ , Gaussian, trapezoidal envelopes); numerical integration only for arbitrary tabulated data
- Implement multiple envelope functions as strategy pattern
- Ensure smooth turn-on/off to avoid high-frequency artifacts

### 3.4.5 Testing Strategy

- Verify  $\vec{E}(t) = -d\vec{A}/dt$  numerically
- Check envelope function:  $f(0) = 0$ ,  $f(T) = 0$  for smooth pulses
- Validate Keldysh parameter  $\gamma$  against analytical values
- Plot field and verify correct period, phase, polarization

### 3.5 Module 5: Propagator Module

#### 3.5.1 Purpose

Implement the split-operator time-propagation scheme in energy representation.

#### 3.5.2 Mathematical Foundation

The split-operator method factorizes the time-evolution operator:

$$\hat{U}(\Delta t) = e^{-i\hat{H}\Delta t} \approx e^{-i\hat{H}_0\Delta t/2} e^{-i\hat{V}(t)\Delta t} e^{-i\hat{H}_0\Delta t/2} + \mathcal{O}(\Delta t^3) \quad (18)$$

**Energy representation propagation:**

$$e^{-i\hat{H}_0\Delta t/2} = \sum_{n,l,m} e^{-iE_{n,l}\Delta t/2} |\Phi_{n,l,m}\rangle \langle \Phi_{n,l,m}| \quad (19)$$

The S-matrix implements this:

$$S_{l,ij} = \sum_n \phi_{n,l}(x_i) \phi_{n,l}(x_j) e^{-iE_{n,l}\Delta t/2} w_j r'_j \quad (20)$$

**Interaction term (length gauge):**

$$\hat{V}(t) = \vec{r} \cdot \vec{E}(t) = r E_z(t) \cos \theta \quad (\text{for } \vec{E} \parallel \hat{z}) \quad (21)$$

**3D transformation sequence:**

$$g_l(r) \xrightarrow{\text{Legendre}} \Psi(r, \theta) \quad (22)$$

$$\Psi(r, \theta) \xrightarrow{\text{exp}} \Psi(r, \theta) e^{-ir E_z(t) \cos \theta \Delta t} \quad (23)$$

$$\Psi(r, \theta) \xrightarrow{\text{Legendre}^{-1}} g'_l(r) \quad (24)$$



### 3.5.3 Interface

#### Propagator Module Interface

##### Inputs:

- $\psi(t)$ : Current wavefunction (radial components  $\{g_l(x_i)\}$ )
- S-matrices:  $\{S_l\}$  from eigenstate data
- $\vec{E}(t)$ : Electric field at current time
- $\Delta t$ : Time step
- Grid and basis function data

##### Outputs:

- $\psi(t + \Delta t)$ : Propagated wavefunction
- Diagnostics: norm, energy, dipole moment

##### Methods:

- `propagate_H0(psi, dt/2)`: Apply  $e^{-i\hat{H}_0\Delta t/2}$
- `propagate_interaction(psi, E, dt)`: Apply  $e^{-i\hat{V}\Delta t}$
- `radial_to_angular(g_l)`: Transform to grid space  $\Psi(r, \theta)$
- `angular_to_radial(Psi)`: Transform back to  $g_l(r)$
- `compute_norm(psi)`: Calculate  $\langle\psi|\psi\rangle$
- `apply_absorber(psi)`: Absorbing boundary conditions

### 3.5.4 Algorithm: One Time Step

### 3.5.5 Implementation Notes

- Pre-compute S-matrices once before time loop
- Store only bound and low-energy continuum states in S-matrix (cutoff at  $E \sim 50$  a.u.)
- Use BLAS for S-matrix multiplication:  $g^{(1)} = S \cdot g$
- Parallelize transformations: radial grid points are independent
- Absorbing boundary:  $\cos^{1/4}$  mask to prevent reflections
- Monitor norm: should remain close to 1.0 (excluding absorbed flux)

### 3.5.6 Testing Strategy

- Field-free evolution: energy should be conserved
- Test with simple field: compare with exact solutions for hydrogen in weak fields
- Verify time-reversal symmetry:  $\psi(T) \rightarrow \psi(0)$  when reversing field

---

**Algorithm 1** Split-Operator Time Propagation

---

```
1: Input:  $\psi(t) = \{g_l(x_i)\}$ ,  $\vec{E}(t)$ ,  $\Delta t$ 
2: Step 1:  $g_l^{(1)} = S_l \cdot g_l$  ▷ Apply  $e^{-i\hat{H}_0\Delta t/2}$ 
3: Step 2: Transform to grid space
4: for each  $r_i$  do
5:   for each  $\theta_j$  do
6:      $\Psi(r_i, \theta_j) = \sum_{l=0}^{l_{max}} g_l^{(1)}(x_i) P_l(\cos \theta_j) / \sqrt{2/(2l+1)}$ 
7:   end for
8: end for
9: Step 3: Apply interaction
10: for each  $r_i$  do
11:   for each  $\theta_j$  do
12:      $\Psi(r_i, \theta_j) \leftarrow \Psi(r_i, \theta_j) \exp(-ir_i E_z(t) \cos \theta_j \Delta t)$ 
13:   end for
14: end for
15: Step 4: Transform back to radial
16: for each  $l$  do
17:   for each  $r_i$  do
18:      $g_l^{(2)}(x_i) = \sum_{j=1}^{N_\theta} \Psi(r_i, \theta_j) P_l(\cos \theta_j) w_\theta(j) \sqrt{(2l+1)/2}$ 
19:   end for
20: end for
21: Step 5:  $g_l^{(3)} = S_l \cdot g_l^{(2)}$  ▷ Apply  $e^{-i\hat{H}_0\Delta t/2}$ 
22: Step 6: Apply absorber:  $g_l(x_i) \leftarrow g_l(x_i) \cos^{1/4}[\pi(r_i - r_0)/(2(r_{max} - r_0))]$  for  $r_i > r_0$ 
23: Output:  $\psi(t + \Delta t) = \{g_l^{(3)}(x_i)\}$ 
```

---

- Check convergence: vary  $\Delta t$ ,  $N_r$ ,  $l_{max}$  and monitor observables
- Validate against published benchmarks (e.g., hydrogen ionization rates)

## 3.6 Module 6: Region Split Module

### 3.6.1 Purpose

Split the wavefunction into inner and outer regions for separate analysis using a smooth transition function.

### 3.6.2 Mathematical Foundation

The smooth splitting function (from PRA 74, 031405):

$$F_s(r) = \frac{1}{1 + e^{-(r-R_c)/\Delta}} \quad (25)$$

Wavefunction decomposition:

$$\Psi_{\text{outer}}(r, \theta, \phi) = F_s(r)\Psi(r, \theta, \phi) \quad (26)$$

$$\Psi_{\text{inner}}(r, \theta, \phi) = [1 - F_s(r)]\Psi(r, \theta, \phi) \quad (27)$$

In radial components:

$$g_{l,\text{outer}}(x_i) = F_s(r_i)g_l(x_i), \quad g_{l,\text{inner}}(x_i) = [1 - F_s(r_i)]g_l(x_i) \quad (28)$$

### 3.6.3 Interface

#### Region Split Module Interface

##### Inputs:

- $\psi(t)$ : Current wavefunction  $\{g_l(x_i)\}$
- $R_c$ : Critical radius separating regions
- $\Delta$ : Smoothness parameter
- Radial grid:  $\{r_i\}$

##### Outputs:

- $\psi_{\text{inner}}$ : Inner region wavefunction
- $\psi_{\text{outer}}$ : Outer region wavefunction
- $i_{\text{split}}$ : Grid index where  $F_s(r_i) > 0.998$  (for efficiency)

##### Methods:

- `compute_split_function(r)`: Evaluate  $F_s(r)$
- `split_wavefunction(psi)`: Decompose into inner/outer
- `get_inner_region()`: Return  $\psi_{\text{inner}}$
- `get_outer_region()`: Return  $\psi_{\text{outer}}$

### 3.6.4 Physical Interpretation

- **Inner region** ( $r < R_c$ ): Contains bound states and near-core continuum

- **Outer region** ( $r > R_c$ ): Free-particle-like behavior, suitable for plane wave projection
- Typical values:  $R_c = 100$  a.u.,  $\Delta = 5$  a.u.
- Choice of  $R_c$ : Should be  $\gtrsim 2\alpha_0$  where  $\alpha_0 = E_0/\omega^2$  is the quiver amplitude

### 3.6.5 Implementation Notes

- Pre-compute  $F_s(r_i)$  once at initialization
- Find  $i_{\text{split}}$  such that  $F_s(r_i) > 0.998$ : only process  $i \geq i_{\text{split}}$  for outer region
- Splitting is performed periodically (every  $N_{\text{split}}$  steps, e.g., every 10 steps)
- Store split function as lookup table

### 3.6.6 Testing Strategy

- Verify:  $|\psi_{\text{inner}}|^2 + |\psi_{\text{outer}}|^2 \approx |\psi|^2$  (neglect  $\mathcal{O}(\Delta^2)$  terms)
- Check normalization:  $\langle \psi_{\text{inner}} | \psi_{\text{inner}} \rangle + \langle \psi_{\text{outer}} | \psi_{\text{outer}} \rangle \approx 1$
- Vary  $R_c$  and  $\Delta$ : results should be stable within reasonable ranges
- Visualize  $F_s(r)$ : should be smooth transition, no discontinuities

## 3.7 Module 7: Continuum Projection Module

### 3.7.1 Purpose

Project outer region wavefunction onto momentum eigenstates (Volkov or Coulomb-Volkov states).

### 3.7.2 Mathematical Foundation

**Volkov States (Program 1):** Plane waves with Volkov phase:

$$\phi_{\vec{p}}^V(r, \theta, \phi) = (2\pi)^{-3/2} e^{i[\vec{p} \cdot \vec{r} - S_{\vec{p}}(t)]} \quad (29)$$

where the Volkov action is:

$$S_{\vec{p}}(t) = \int_0^t \left[ \frac{p^2}{2} + \vec{p} \cdot \vec{A}(t') + \frac{A^2(t')}{2} \right] dt' \quad (30)$$

Expansion in spherical harmonics:

$$e^{i\vec{p} \cdot \vec{r}} = 4\pi \sum_{l=0}^{\infty} \sum_{m=-l}^l i^l j_l(pr) Y_l^{m*}(\hat{p}) Y_l^m(\hat{r}) \quad (31)$$

**Coulomb-Volkov States (Program 2):** Coulomb continuum functions with Coulomb phase shift  $\delta_l(p)$ :

$$\phi_{\vec{p}}^{CV}(r, \theta, \phi) = \sqrt{\frac{2}{\pi}} \sum_{l,m} i^l e^{i\delta_l(p)} F_l(\eta, pr) Y_l^m(\hat{r}) Y_l^{m*}(\hat{p}) \quad (32)$$

where  $F_l(\eta, pr)$  is the regular Coulomb wave function with Sommerfeld parameter  $\eta = -Z/p$ .

**Projection:**

$$C_{l,m}(p) = \langle \phi_{p,l,m} | \psi_{\text{outer}} \rangle = \int \phi_{p,l,m}^*(r, \theta, \phi) \psi_{\text{outer}}(r, \theta, \phi) r^2 dr d\Omega \quad (33)$$

For Volkov (spherical Bessel):

$$C_{l,m}(p) = i^{-l} \sqrt{\frac{2}{\pi}} \int_0^{\infty} j_l(pr) g_{l,\text{outer}}(r) r^2 dr \quad (34)$$

For Coulomb-Volkov:

$$C_{l,m}(p) = \frac{i^{-l}}{\pi} e^{i\delta_l(p)} \int_0^{\infty} F_l(\eta, pr) g_{l,\text{outer}}(r) r^2 dr \quad (35)$$

### 3.7.3 Interface

#### Continuum Projection Module Interface

##### Inputs:

- $\psi_{\text{outer}}$ : Outer region wavefunction from Region Split
- Momentum grid:  $\{p_k\}$ ,  $k = 1, \dots, N_p$
- Continuum state type: Volkov or Coulomb-Volkov
- Radial grid:  $\{r_i\}$ , weights:  $\{w_i\}$
- For Coulomb-Volkov: charge  $Z$ , phase shifts  $\{\delta_l(p)\}$

##### Outputs:

- $\{C_{l,m}(p_k)\}$ : Momentum amplitudes
- Momentum distribution:  $P(p) = \sum_{l,m} |C_{l,m}(p)|^2$

##### Methods:

- `compute_bessel_functions(p, r)`: Spherical Bessel  $j_l(pr)$
- `compute_coulomb_functions(p, r)`: Regular Coulomb  $F_l(\eta, pr)$
- `compute_phase_shifts(p, l)`: Coulomb phases  $\delta_l(p)$
- `project_onto_momentum(psi_outer, p)`: Calculate  $C_{l,m}(p)$
- `compute_momentum_distribution()`: Sum over  $l, m$  to get  $P(p)$

### 3.7.4 Algorithm: Momentum Projection

---

#### Algorithm 2 Projection onto Continuum States

---

- 1: **Input:**  $\psi_{\text{outer}} = \{g_{l,\text{outer}}(x_i)\}$ , momentum grid  $\{p_k\}$
  - 2: Pre-compute continuum radial functions:  $\{j_l(p_k r_i)\}$  or  $\{F_l(\eta, p_k r_i)\}$
  - 3: **for**  $l = 0$  to  $l_{\text{max}}$  **do**
  - 4:   **for**  $k = 1$  to  $N_p$  **do**
  - 5:     Initialize  $C_l(p_k) = 0$
  - 6:     **for**  $i = i_{\text{split}}$  to  $N_r$  **do**
  - 7:        $C_l(p_k) \leftarrow C_l(p_k) + \text{continuum}_l(p_k, r_i) \cdot g_{l,\text{outer}}(x_i) \cdot w_i r_i^2 \sqrt{r_i'}$
  - 8:     **end for**
  - 9:      $C_l(p_k) \leftarrow C_l(p_k) \cdot i^{-l} \sqrt{2/\pi}$  ▷ Normalization
  - 10:    **if** Coulomb-Volkov **then**
  - 11:       $C_l(p_k) \leftarrow C_l(p_k) \cdot e^{i\delta_l(p_k)}$  ▷ Phase shift
  - 12:    **end if**
  - 13:   **end for**
  - 14: **end for**
  - 15: Compute  $P(p_k) = \sum_{l=0}^{l_{\text{max}}} (2l+1) |C_l(p_k)|^2$
  - 16: **Output:**  $\{C_l(p_k)\}$ ,  $P(p_k)$
-

### 3.7.5 Implementation Notes

- Pre-compute and cache Bessel/Coulomb functions: expensive to evaluate
- Use recurrence relations for spherical Bessel functions to improve efficiency
- Coulomb wave functions: use library routines (e.g., GSL, Coulomb90, or implement Algorithm 292)
- Parallelization: independent over  $l$  and  $p$
- For linearly polarized fields:  $m = 0$  is sufficient due to cylindrical symmetry

### 3.7.6 Testing Strategy

- Normalization check:  $\int |P(p)|^2 p^2 dp \approx \langle \psi_{\text{outer}} | \psi_{\text{outer}} \rangle$
- Compare Volkov vs. Coulomb-Volkov: should differ mainly near low momenta
- Test with known wavefunctions (e.g., Gaussian wave packets)
- Validate Coulomb phase shifts against analytical formulas
- Convergence: increase  $l_{\text{max}}$  and check  $P(p)$  stabilization

### 3.8 Module 8: Observable Module

#### 3.8.1 Purpose

Extract physical observables from the wavefunction during and after time evolution.

#### 3.8.2 Mathematical Foundation

**Norm:**

$$N(t) = \langle \psi(t) | \psi(t) \rangle = \sum_{l=0}^{l_{max}} \sum_{i=1}^{N_r} |g_l(x_i, t)|^2 w_i r'_i \quad (36)$$

**Population in Bound States:**

$$P_{n,l}(t) = |\langle \Phi_{n,l} | \psi(t) \rangle|^2 = \left| \sum_{i=1}^{N_r} \phi_{n,l}(x_i) g_l(x_i, t) w_i r'_i \right|^2 \quad (37)$$

**Ionization Probability:**

$$P_{\text{ion}}(t) = 1 - \sum_{n,l,m} P_{n,l}(t) \quad (38)$$

where sum is over all bound states ( $E_{n,l} < 0$ ).

**Dipole Moment (for HHG):**

$$d(t) = \langle \psi(t) | \hat{z} | \psi(t) \rangle = \sum_{l,l'} \int g_l^*(r, t) \cdot r \cdot g_{l'}(r, t) \cdot \langle Y_l^m | \cos \theta | Y_{l'}^{m'} \rangle r^2 dr \quad (39)$$

For dipole acceleration (used in programs):

$$\ddot{d}(t) = \langle \psi(t) | \frac{\partial V}{\partial z} | \psi(t) \rangle \quad (40)$$

**HHG Spectrum:**

$$S(\omega) = \left| \int_0^T \ddot{d}(t) e^{i\omega t} dt \right|^2 \quad (41)$$

**Photoelectron Momentum Distribution:** From Continuum Projection Module:

$$P(\vec{p}) = |C_{\text{total}}(\vec{p})|^2 = \left| \sum_{l,m} C_{l,m}(p) Y_l^m(\hat{p}) \right|^2 \quad (42)$$



### 3.8.3 Interface

#### Observable Module Interface

##### Inputs:

- $\psi(t)$ : Current wavefunction
- Eigenstates:  $\{\Phi_{n,l}\}, \{E_{n,l}\}$
- Grid and weights
- Observable type: norm, populations, dipole, momentum distribution

##### Outputs:

- Time-dependent observables:  $N(t), P_{n,l}(t), d(t), \ddot{d}(t)$
- Final observables:  $P(\vec{p}), S(\omega)$

##### Methods:

- `compute_norm(psi)`: Calculate  $\langle\psi|\psi\rangle$
- `compute_populations(psi)`: Project onto bound states
- `compute_ionization_probability(psi)`:  $1 - \sum P_{n,l}$
- `compute_dipole_moment(psi)`: Evaluate  $\langle z \rangle$
- `compute_dipole_acceleration(psi)`: Evaluate  $\langle \partial V / \partial z \rangle$
- `compute_hhg_spectrum(dipole_history)`: FFT of  $\ddot{d}(t)$
- `compute_momentum_distribution(C_lm)`:  $P(\vec{p})$  from continuum amplitudes

### 3.8.4 Implementation Notes

- Store time histories: create buffers for  $d(t), \ddot{d}(t), P_{n,l}(t)$
- FFT for HHG: use FFTW or similar library, apply windowing function to reduce spectral leakage
- Dipole matrix elements: pre-compute  $\langle Y_l^m | \cos \theta | Y_{l'}^{m'} \rangle$  using selection rules
- Selection rules:  $l' = l \pm 1, m' = m$  for  $z$  operator
- Monitor norm during evolution: should decrease due to absorption but remain  $> 0.9$  typically

### 3.8.5 Testing Strategy

- Initial norm:  $N(0) = 1$  for normalized initial state
- Field-free case: populations  $P_{n,l}$  should remain constant
- Symmetry checks: for linearly polarized fields, dipole should be along field axis
- HHG spectrum: verify odd harmonics only (for symmetric systems and fields)
- Cutoff law:  $\omega_{\max} \approx I_p + 3.17U_p$

## 3.9 Module 9: I/O Module

### 3.9.1 Purpose

Handle input parameter files and output data files in a structured format.

### 3.9.2 Input Parameters

Create a configuration file format (e.g., JSON, YAML, or TOML):

```
1 {
2   "system": {
3     "atom": "He",
4     "Z_eff": 1.4,
5     "r_D": 17.85,
6     "ground_state": {"n": 1, "l": 0, "E": -0.9}
7   },
8   "grid": {
9     "N_r": 600,
10    "N_theta": 180,
11    "N_phi": 60,
12    "l_max": 99,
13    "r_max": 200.0,
14    "L": 150.0,
15    "alpha": 3.5
16  },
17  "field": {
18    "E0": 0.08,
19    "wavelength_nm": 800,
20    "CEP_phase": 0.0,
21    "polarization": [0.0, 0.0, 1.0],
22    "envelope": "trapezoidal",
23    "duration_cycles": 10,
24    "ramp_cycles": 3
25  },
26  "propagation": {
27    "dt": 0.1,
28    "t_max": "auto",
29    "absorber_start": 180.0,
30    "region_split": {
31      "enabled": true,
32      "R_c": 100.0,
33      "Delta": 5.0,
34      "interval_steps": 10
35    }
36  },
37  "observables": {
38    "calculate_ionization": true,
39    "calculate_hhg": true,
40    "momentum_grid": {
41      "p_max": 4.5,
42      "N_p": 600
43    },
44    "bound_state_tracking": [1, 2, 3]
45  },
46  "output": {
47    "directory": "./output",
48    "save_wavefunction_interval": 100,
49    "save_populations": true,
50    "save_dipole": true,
51    "save_momentum_distribution": true
52  }
```

## Listing 1: Example Input Configuration (JSON format)

### 3.9.3 Output Files

**File Structure:**

- `parameters.json`: Copy of input parameters
- `field.dat`: Time grid and electric field values
- `populations.dat`: Bound state populations vs. time
- `dipole.dat`: Dipole moment/acceleration vs. time
- `hhg_spectrum.dat`: HHG spectrum  $S(\omega)$
- `momentum_distribution.dat`: Final  $P(\vec{p})$
- `wavefunction_tXXXX.h5`: Wavefunction snapshots (HDF5 format)

### 3.9.4 Interface

**I/O Module Interface****Methods:**

- `read_input_file(filename)`: Parse configuration
- `write_parameters(params)`: Save parameters to output
- `write_field_data(time, E)`: Save electric field
- `write_populations(time, P_nl)`: Save populations
- `write_dipole(time, d, ddot_d)`: Save dipole data
- `write_hhg_spectrum(omega, S)`: Save HHG spectrum
- `write_momentum_distribution(p, P_p)`: Save momentum dist.
- `write_wavefunction(psi, step)`: Save wavefunction snapshot
- `create_output_directory()`: Create directory structure

### 3.9.5 Implementation Notes

- Use binary formats (HDF5, NetCDF) for large datasets (wavefunctions)
- Include metadata: timestamp, code version, parameter checksums
- Atomic units by default; provide conversion factors in metadata
- Implement checkpoint/restart: save state periodically for long runs
- Validate input parameters: check physical bounds, consistency

### 3.9.6 Testing Strategy

- Round-trip test: write and read back, compare
- Parse various input formats: test error handling
- Large file test: ensure memory efficiency for wavefunction I/O
- Restart test: verify continuation from checkpoint

## **3.10 Module 10: Orchestrator Module**

### **3.10.1 Purpose**

Coordinate the execution flow, connecting all modules to perform complete ionization or HHG calculations.

### 3.10.2 Main Execution Flow

---

**Algorithm 3** Main Orchestrator: TDSE Calculation

---

```

1: Initialization Phase
2: Read input parameters from file
3: Initialize Grid Module  $\rightarrow$  obtain grids, weights, basis functions
4: Initialize Hamiltonian Module  $\rightarrow$  construct  $\{H_l\}$ 
5: Initialize Eigenstate Module  $\rightarrow$  solve eigensystems, get  $\{E_{n,l}\}, \{\Phi_{n,l}\}$ 
6: Construct S-matrices for propagation
7: Initialize Field Module  $\rightarrow$  set up field parameters (no pre-computation needed)
8: Initialize initial state  $\psi(0) = \Phi_{n_0,l_0,m_0}$  (e.g., ground state)
9: Initialize output files
10:
11: Time Evolution Loop
12: for  $m = 1$  to  $N_{\text{steps}}$  do
13:    $t = m \cdot \Delta t$ 
14:   Call Propagator:  $\psi(t) \rightarrow \psi(t + \Delta t)$ 
15:
16:   if  $m \bmod N_{\text{obs}} = 0$  then
17:      $\triangleright$  Compute observables periodically Call Observable Module: compute norm,
18:     populations, dipole Write to output files
19:   end if
20:
21:   if  $m \bmod N_{\text{split}} = 0$  and region splitting enabled then
22:     Call Region Split Module:  $\psi \rightarrow \psi_{\text{inner}} + \psi_{\text{outer}}$ 
23:     Call Continuum Projection Module:  $\psi_{\text{outer}} \rightarrow \{C_{l,m}(p)\}$ 
24:     Accumulate momentum distributions
25:   end if
26:
27:   if  $m \bmod N_{\text{checkpoint}} = 0$  then
28:     Save checkpoint for restart capability
29:   end if
30: end for
31:
32: Post-Processing Phase
33: if Calculate Ionization then
34:   Compute final momentum distribution  $P(\vec{p})$ 
35:   Project onto Cartesian momentum grids  $(p_x, p_y, p_z)$ 
36:   Write momentum distributions to file
37: end if
38:
39: if Calculate HHG then
40:   Retrieve dipole acceleration history  $\ddot{d}(t)$ 
41:   Apply windowing function
42:   Perform FFT:  $\ddot{d}(t) \rightarrow \ddot{d}(\omega)$ 
43:   Compute HHG spectrum:  $S(\omega) = |\ddot{d}(\omega)|^2$ 
44:   Write HHG spectrum to file
45: end if
46:
47: Finalize output files
48: Print summary statistics

```

---

### 3.10.3 Interface

#### Orchestrator Module Interface

##### Methods:

- `run_calculation(input_file)`: Main entry point
- `initialize_modules()`: Set up all modules
- `execute_time_evolution()`: Main time loop
- `post_process_results()`: Extract final observables
- `print_summary()`: Display calculation summary

### 3.10.4 Error Handling

- Validate input parameters before starting calculation
- Monitor norm during evolution: warn if  $N(t) < 0.5$  (excessive absorption)
- Check energy conservation in field-free regions
- Catch numerical instabilities (NaN, Inf) and abort gracefully
- Provide informative error messages with suggestions for fixes

### 3.10.5 Performance Optimization

- Parallelize over independent operations:  $l$  blocks, grid points
- Use vectorized operations (SIMD) where possible
- Profile code to identify bottlenecks: S-matrix multiplication, transformations
- Consider GPU acceleration for transformations (if applicable)
- Implement memory pooling to reduce allocations in time loop

## 4 Integration Plan

### 4.1 Phase 1: Foundation Layer (Weeks 1-2)

1. Implement Grid Module
  - Legendre zero finding
  - Algebraic mapping
  - Quadrature weights
  - Basis function evaluation
2. Implement Hamiltonian Module
  - Kinetic energy matrix
  - Potential energy (multiple models)
  - Centrifugal barrier
3. Test: Verify eigenvalues match known atomic data

### 4.2 Phase 2: Preparation Layer (Weeks 3-4)

1. Implement Eigenstate Module
  - Interface with LAPACK for diagonalization
  - Eigenstate normalization
  - State classification
2. Implement Field Module
  - Envelope functions
  - Electric field and vector potential
  - Parameter derivation ( $U_p, \gamma$ )
3. Test: Compare eigenstates with literature, validate field properties

### 4.3 Phase 3: Evolution Layer (Weeks 5-7)

1. Implement Propagator Module
  - S-matrix construction
  - Split-operator steps
  - 3D transformations (radial  $\leftrightarrow$  angular)
  - Absorbing boundary conditions
2. Implement Region Split Module
  - Smooth transition function
  - Inner/outer decomposition
3. Test: Field-free evolution (energy conservation), weak-field perturbation theory



#### 4.4 Phase 4: Analysis Layer (Weeks 8-9)

1. Implement Observable Module
  - Norm, populations
  - Dipole moment/acceleration
  - HHG spectrum via FFT
2. Implement Continuum Projection Module
  - Spherical Bessel functions
  - Coulomb wave functions
  - Momentum projection
3. Test: Compare momentum distributions with benchmarks, validate HHG cutoff law

#### 4.5 Phase 5: Application Layer (Week 10)

1. Implement I/O Module
  - Configuration parser
  - Output file writers
  - Checkpoint/restart
2. Implement Orchestrator Module
  - Main execution flow
  - Error handling
  - Performance profiling
3. Test: Full end-to-end calculations for ionization and HHG

#### 4.6 Phase 6: Validation & Optimization (Weeks 11-12)

1. Validate against published benchmarks
  - Hydrogen strong-field ionization (TDSE benchmarks)
  - Helium HHG spectra (comparison with experiments)
  - ATI photoelectron spectra
2. Performance optimization
  - Profile critical sections
  - Parallelize hot loops
  - Memory access optimization
3. Documentation
  - API documentation
  - User guide
  - Example calculations

## 5 Testing Strategy

### 5.1 Unit Tests

Each module should have comprehensive unit tests:

- **Grid Module:** Orthogonality, quadrature accuracy
- **Hamiltonian Module:** Hermiticity, eigenvalue validation
- **Eigenstate Module:** Orthonormality, energy convergence
- **Field Module:** Maxwell equations, energy conservation
- **Propagator Module:** Norm conservation, reversibility
- **Observable Module:** Sum rules, selection rules

### 5.2 Integration Tests

- Field-free evolution: verify energy conservation
- Weak-field perturbation: compare with analytical perturbation theory
- Strong-field tunneling: compare ionization rates with ADK/PPT theory
- HHG spectrum: verify cutoff law and harmonic structure

### 5.3 Regression Tests

Maintain a suite of benchmark calculations:

- Hydrogen in 800 nm,  $5 \times 10^{14}$  W/cm<sup>2</sup> field
- Helium HHG at various intensities
- ATI momentum distributions

Store reference results and automatically compare new code outputs.

### 5.4 Performance Tests

- Measure time per propagation step
- Profile memory usage
- Strong scaling: vary number of cores
- Weak scaling: increase problem size

## 6 Numerical Considerations

### 6.1 Numerical Precision

- Use double precision (Float64) by default
- Critical sections may need quad precision (Float128): phase calculations, small energy differences
- Accumulation of small terms: use Kahan summation or compensated summation

### 6.2 Stability Criteria

**Time step:**

$$\Delta t \lesssim \frac{\pi}{\max(E_{n,l})} \quad (43)$$

Typically  $\Delta t \sim 0.05 - 0.1$  a.u.

**Grid spacing:**

$$\Delta r \lesssim \frac{\pi}{p_{\max}}, \quad p_{\max} \approx \sqrt{2(I_p + 10U_p)} \quad (44)$$

**Angular momentum convergence:**

$$l_{\max} \gtrsim k_0 r_{\max} \quad (45)$$

where  $k_0 = \sqrt{2I_p}$ .

### 6.3 Convergence Parameters

Systematically vary:

- $N_r$ : Radial grid points (400 – 1000)
- $l_{\max}$ : Maximum angular momentum (50 – 200)
- $r_{\max}$ : Box size (150 – 300 a.u.)
- $\Delta t$ : Time step (0.05 – 0.2 a.u.)
- $E_{\text{cutoff}}$ : Energy cutoff for S-matrix (30 – 100 a.u.)

Monitor observable convergence: aim for  $< 1\%$  variation.

### 6.4 Parallelization Strategy

**Coarse-grained parallelism:**

- Each  $l$  block is independent: distribute over MPI ranks or threads
- Each momentum  $p_k$  is independent: parallelize projection

**Fine-grained parallelism:**

- Matrix multiplications: use multi-threaded BLAS
- Transformations: parallelize loops over grid points
- FFTs: use FFTW with threading

**Hybrid MPI+OpenMP:**

- MPI over  $l$  blocks
- OpenMP within each  $l$  block for transformations

## 7 Code Structure Example

### 7.1 Directory Layout

```
tdse_refactor/  
  src/  
    grid/  
      legendre_grid.jl (or .py, .cpp)  
      angular_grid.jl  
      basis_functions.jl  
    hamiltonian/  
      kinetic.jl  
      potential.jl  
      hamiltonian.jl  
    eigenstate/  
      solver.jl  
      eigenstates.jl  
    field/  
      envelope.jl  
      laser_field.jl  
    propagation/  
      smatrix.jl  
      split_operator.jl  
      transformation.jl  
      absorber.jl  
    region_split/  
      splitting.jl  
    continuum/  
      bessell.jl  
      coulomb.jl  
      projection.jl  
    observables/  
      populations.jl  
      dipole.jl  
      momentum.jl  
      hhg.jl  
    io/  
      input.jl  
      output.jl  
    orchestrator/  
      main.jl  
  tests/  
    test_grid.jl  
    test_hamiltonian.jl  
    ...  
    test_integration.jl  
  benchmarks/  
    hydrogen_ionization.jl  
    helium_hhg.jl  
  docs/  
    user_guide.md  
    api_reference.md
```

```

examples/
  example_ionization.json
  example_hhg.json
  README.md

```

## 7.2 Example Module Interface (Julia)

```

1 module GridModule
2
3 export LegendreGrid, compute_grid, get_radial_grid, get_angular_grid
4
5 struct LegendreGrid
6     Nr::Int
7     Ntheta::Int
8     L::Float64
9     alpha::Float64
10    rmax::Float64
11
12    x::Vector{Float64}           # Legendre points
13    r::Vector{Float64}           # Mapped radial grid
14    rprime::Vector{Float64}      # dr/dx
15    w::Vector{Float64}           # Weights
16
17    theta::Vector{Float64}       # Angular grid
18    wtheta::Vector{Float64}      # Angular weights
19
20    Pn::Matrix{Float64}          # Legendre polynomials
21    Pn_prime::Matrix{Float64}    # Derivatives
22    Plm::Array{Float64, 3}       # Associated Legendres
23 end
24
25 function compute_grid(Nr::Int, Ntheta::Int, L::Float64,
26                      alpha::Float64, rmax::Float64, lmax::Int)
27     # Implementation
28     # Returns LegendreGrid object
29 end
30
31 # Accessor functions
32 get_radial_grid(grid::LegendreGrid) = grid.r
33 get_angular_grid(grid::LegendreGrid) = grid.theta
34
35 end # module

```

Listing 2: Grid Module Interface Example

## 8 Mathematical Reference

### 8.1 Key Equations Summary

#### Core TDSE Equations

##### Time-Dependent Schrödinger Equation:

$$i\frac{\partial\Psi}{\partial t} = \left[\hat{H}_0 + \vec{r} \cdot \vec{E}(t)\right] \Psi \quad (46)$$

##### Split-Operator Approximation:

$$\Psi(t + \Delta t) \approx e^{-i\hat{H}_0\Delta t/2} e^{-i\hat{V}(t)\Delta t} e^{-i\hat{H}_0\Delta t/2} \Psi(t) \quad (47)$$

##### Energy Representation:

$$e^{-i\hat{H}_0\Delta t/2} = \sum_{n,l,m} e^{-iE_{n,l}\Delta t/2} |\Phi_{n,l,m}\rangle \langle \Phi_{n,l,m}| \quad (48)$$

##### Coordinate Mapping:

$$r(x) = L \frac{1+x}{1-x+\alpha}, \quad \frac{dr}{dx} = \frac{2L(1+\alpha)}{(1-x+\alpha)^2} \quad (49)$$

##### Region Splitting Function:

$$F_s(r) = \frac{1}{1 + \exp[-(r - R_c)/\Delta]} \quad (50)$$

##### HHG Spectrum:

$$S(\omega) = \left| \mathcal{F} \left\{ \frac{d^2 d(t)}{dt^2} \right\} \right|^2 \quad (51)$$

##### Momentum Distribution:

$$P(\vec{p}) = \left| \langle \phi_{\vec{p}} | \Psi(T) \rangle \right|^2 \quad (52)$$

### 8.2 Physical Constants (Atomic Units)

- $\hbar = m_e = e = 1$
- $a_0 = 0.529 \text{ \AA}$  (Bohr radius)
- Time: 1 a.u. = 24.2 as
- Energy: 1 a.u. = 27.2 eV (Hartree)
- Electric field: 1 a.u. =  $5.14 \times 10^{11} \text{ V/m}$
- Intensity: 1 a.u. =  $3.51 \times 10^{16} \text{ W/cm}^2$

### 8.3 Useful Relations

$$\lambda(\text{nm}) = \frac{45.56}{\omega(\text{a.u.})} \quad (53)$$

$$U_p(\text{eV}) = 9.33 \times 10^{-14} I(\text{W/cm}^2) \lambda^2(\mu\text{m}^2) \quad (54)$$

$$\gamma = \omega \sqrt{\frac{2I_p}{E_0^2}} \quad (55)$$

## 9 Recommended Libraries and Tools

### 9.1 Julia Ecosystem

- **LinearAlgebra.jl**: Built-in BLAS/LAPACK
- **SpecialFunctions.jl**: Bessel functions, error functions
- **FastGaussQuadrature.jl**: Gauss-Legendre quadrature
- **FFTW.jl**: Fast Fourier transforms
- **HDF5.jl**: Binary I/O for large datasets
- **DifferentialEquations.jl**: ODE solvers (for Coulomb functions)
- **Plots.jl** / **Makie.jl**: Visualization
- **BenchmarkTools.jl**: Performance testing
- **Test.jl**: Unit testing framework
- **Documenter.jl**: Documentation generation

### 9.2 Python Ecosystem

- **NumPy** / **SciPy**: Core numerical computing
- **mpmath**: Arbitrary precision arithmetic
- **scipy.special**: Special functions (Bessel, Coulomb via ‘coulombf’)
- **numba**: JIT compilation for performance
- **h5py**: HDF5 I/O
- **matplotlib** / **plotly**: Visualization
- **pytest**: Testing framework
- **sphinx**: Documentation

### 9.3 C++ Ecosystem

- **Eigen** / **Armadillo**: Linear algebra
- **GSL (GNU Scientific Library)**: Special functions, quadrature
- **MKL** / **OpenBLAS**: Optimized BLAS/LAPACK
- **FFTW**: FFTs
- **HDF5**: Binary I/O
- **Boost**: Utilities, special functions
- **OpenMP** / **TBB**: Parallelization
- **Google Test**: Unit testing
- **Doxygen**: Documentation



## 10 Migration Path from Fortran

### 10.1 Step-by-Step Migration

1. **Extract constants and parameters** from Fortran code into configuration files
2. **Identify data structures:** arrays in Fortran → structs/classes in target language
3. **Modularize Fortran code** first (if not already done): create subroutines for each module
4. **Translate module by module:**
  - (a) Start with Grid Module (no dependencies)
  - (b) Then Hamiltonian Module (depends on Grid)
  - (c) Continue in dependency order
5. **Validate each module:** compare outputs with Fortran reference
6. **Integrate modules:** connect via Orchestrator
7. **Full validation:** compare final results (ionization rates, HHG spectra)

### 10.2 Fortran-to-Target Language Mapping

| Fortran         | Julia              | Python            | C++                     |
|-----------------|--------------------|-------------------|-------------------------|
| real*8          | Float64            | np.float64        | double                  |
| complex*16      | ComplexF64         | np.complex128     | std::complex<double>    |
| dimension(:, :) | Matrix             | np.ndarray        | Eigen::MatrixXd         |
| do i=1,n        | for i in 1:n       | for i in range(n) | for (int i=0; i<n; i++) |
| allocate()      | Array{T}(undef, n) | np.empty(n)       | new / std::vector       |
| call sub()      | sub()              | sub()             | sub();                  |
| module          | module             | module/class      | namespace/class         |

### 10.3 Common Pitfalls

- **Array indexing:** Fortran is 1-based, C/C++/Python are 0-based, Julia is 1-based
- **Array layout:** Fortran is column-major, C/C++ are row-major, NumPy can be either
- **Complex number conventions:** Fortran uses `dcmplx`, Julia `ComplexF64`, Python `complex`
- **Matrix multiplication:** Use `*` in Julia, `@` in NumPy, `*` in Eigen
- **Pass by reference:** Fortran defaults to pass-by-reference, other languages vary

## 11 Example Workflow

### 11.1 Ionization Calculation

1. Create input file: `he_ionization.json`
2. Run: `julia main.jl he_ionization.json`
3. Code executes:
  - Read parameters
  - Build grid and Hamiltonian
  - Solve eigensystem
  - Construct S-matrices
  - Initialize ground state
  - Time evolution loop (with periodic region splitting)
  - Project outer region to momentum space
  - Save momentum distributions
4. Output files:
  - `momentum_distribution.dat`:  $P(p_x, p_y, p_z)$
  - `populations.dat`:  $P_{n,l}(t)$
5. Post-process: Plot 2D momentum distributions, analyze ATI peaks

### 11.2 HHG Calculation

1. Create input file: `he_hhg.json`
2. Run: `julia main.jl he_hhg.json`
3. Code executes:
  - Same initialization as ionization
  - Time evolution with dipole moment calculation at each step
  - Store  $d(t)$  and  $\ddot{d}(t)$
  - After evolution: FFT of dipole acceleration
  - Compute HHG spectrum
4. Output files:
  - `dipole.dat`: Time-dependent dipole
  - `hhg_spectrum.dat`:  $S(\omega)$
5. Post-process: Plot HHG spectrum, identify harmonics, verify cutoff

## 12 Future Extensions

### 12.1 Potential Enhancements

- **Multi-electron systems:** Configuration interaction, time-dependent DFT
- **Molecules:** Non-spherical potentials, fixed-nuclei approximation
- **Two-color fields:** Fundamental + harmonic, control of electron trajectories
- **Chirped pulses:** Time-dependent frequency
- **Attosecond pulses:** Short XUV pulses for pump-probe experiments
- **Non-dipole effects:** Magnetic field coupling at relativistic intensities
- **Adaptive grids:** Dynamic mesh refinement during evolution
- **GPU acceleration:** CUDA/ROCm for massive parallelism
- **Machine learning:** Surrogate models for fast parameter scans

### 12.2 Code Organization for Extensions

Design interfaces to allow:

- Pluggable potential models (via strategy pattern)
- Flexible field configurations (multi-color, arbitrary polarization)
- Alternative propagation schemes (e.g., Runge-Kutta, Magnus expansion)
- Different continuum representations (Coulomb, Coulomb-Volkov, Siegert states)

## 13 Conclusion

This refactoring plan provides a comprehensive blueprint for transforming the monolithic Fortran TDSE codes into a modern, modular architecture. The proposed design:

- **Separates concerns:** Each module has a well-defined responsibility
- **Enables reuse:** Modules can be independently tested and reused
- **Facilitates maintenance:** Clear interfaces simplify debugging and updates
- **Supports extensibility:** New features can be added with minimal disruption
- **Preserves performance:** Parallelization and numerical efficiency are maintained
- **Unifies calculations:** Single codebase handles both ionization and HHG

The modular architecture follows the natural flow of TDSE calculations:

Grid  $\rightarrow$  Hamiltonian  $\rightarrow$  Eigenstates  $\rightarrow$  Propagation  $\rightarrow$  Observables

By following this plan, the refactored code will be:

- More readable and maintainable
- Easier to validate and test
- Better documented
- More flexible for research needs
- Accessible to new users

**This refactoring plan is ready for implementation.**

*Each module can be developed and tested independently before integration.*